

RetrievalAttention: ACCELERATING LONG-CONTEXT LLM INFERENCE VIA VECTOR RETRIEVAL

Di Liu^{◇*}, Meng Chen^{♣*}, Baotong Lu, Huiqiang Jiang, Zhenhua Han, Qianxi Zhang, Qi Chen, Chengruidong Zhang, Bailu Ding, Kai Zhang[♣], Chen Chen[◇], Fan Yang, Yuqing Yang, Lili Qiu
Microsoft Research, [◇]Shanghai Jiao Tong University, [♣]Fudan University
{baotonglu, hjiang, qiazh, yuqyang}@microsoft.com

ABSTRACT

Transformer-based Large Language Models (LLMs) have become increasingly important. However, due to the quadratic time complexity of attention computation, scaling LLMs to longer contexts incurs extremely slow inference speed and high GPU memory consumption for caching key-value (KV) vectors. This paper proposes **RetrievalAttention**, a training-free approach to both accelerate attention computation and reduce GPU memory consumption. By leveraging the dynamic sparsity of attention mechanism, RetrievalAttention proposes to build approximate nearest neighbor search (ANNS) indexes for KV vectors in CPU memory and retrieve the most relevant ones through vector search during generation. Unfortunately, we observe that the off-the-shelf ANNS indexes are often ineffective for such retrieval tasks due to the out-of-distribution (OOD) between query vectors and key vectors in the attention mechanism. RetrievalAttention addresses the OOD challenge by designing an attention-aware vector search algorithm that can adapt to the distribution of query vectors. Our evaluation demonstrates that RetrievalAttention achieves near full attention accuracy while only requiring access to 1–3% of the data. This leads to a significant reduction in the inference cost of long-context LLMs, with a much lower GPU memory footprint. In particular, RetrievalAttention only needs a single NVIDIA RTX4090 (24GB) to serve 128K tokens for LLMs with 8B parameters, which is capable of generating one token in 0.188 seconds.

1 INTRODUCTION

Recent transformer-based Large Language Models (Vaswani et al., 2017) have shown remarkable capabilities in processing long contexts. For instance, Gemini 1.5 Pro (Team, 2024) has supported the context window of up to 10 million tokens. While this is promising for analyzing extensive data, supporting longer context windows also introduces challenges for inference efficiency due to the quadratic complexity of attention computation. To enhance efficiency, KV caching, a technique that retains past key and value vectors, has been widely adopted to prevent redundant computations. However, KV caching-based systems face two primary issues: (a) substantial GPU memory requirements, particularly for long contexts, e.g., the Llama-3-8B model requires approximately 125GB per million tokens; and (b) inference latency increases linearly to the context size, primarily due to the time needed to access cached tokens — a common issue across various computing devices, including GPUs. Therefore, reducing storage costs and token access is vital for enhancing inference efficiency.

The solution lies in leveraging the dynamic sparsity inherent in the attention mechanism (Deng et al., 2024). This refers to the phenomenon where each query vector significantly interacts with only a limited subset of key and value vectors, with the selection of these critical vectors varying dynamically based on individual queries. Prior work (Tang et al., 2024; Xiao et al., 2024a; Ribar et al., 2024; Lee et al., 2024; Singhanian et al., 2024) has proposed various techniques to capitalize on this observation to improve the efficiency of attention computation. However, most of these methods identify important tokens either statically (Xiao et al., 2024b; Li et al., 2024) or heuristically (Xiao et al., 2024a; Ribar et al., 2024; Tang et al., 2024), leading to imprecise approximations that often result in a significant performance drop.

*Work during internship at Microsoft.

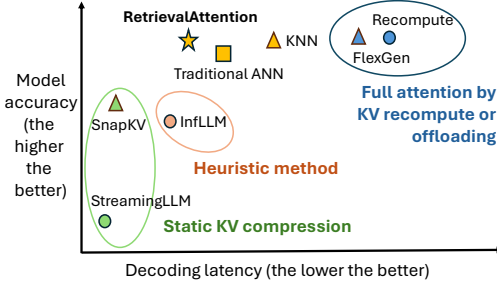


Figure 1: RetrievalAttention achieves similar task accuracy as full attention but exhibits extremely low decoding latency.

Prompt Length	128K	256K	512K	1M
Total Latency (s)	32.8	111	465	1,765
FFN (s)	7.6	15	31	70
Attention (s)	25.2	96	434	1,695
GPU Memory KV Cache (GB)	15.6	31.2	62.5	125

Table 1: Decoding latency and memory required for KV cache of Llama-3-8B across different context lengths on one A100 GPU.

We observe that the Approximate Nearest Neighbor Search (ANNS) index, such as proximity graph (Malkov & Yashunin, 2018), is particularly effective in this context. ANNS index is used to efficiently find the most similar vectors to the query and is widely adopted in various domains like information retrieval (Xiong et al., 2021) and recommendation systems (Cost & Salzberg, 1993; Covington et al., 2016; Pal et al., 2020). When using the inner product as the similarity measurement to build the index for key vectors, searching over the index with the query vector exactly aligns with the attention mechanism.* It can directly identify the most critical key vectors with the maximum inner product to the query vector in sub-linear time complexity, yielding a higher accuracy compared to previous static or heuristic methods (as illustrated in Figure 1). Furthermore, most ANNS algorithms are compatible with CPU implementation, which enables strategic allocation of GPU and CPU memory resources and thus facilitates the handling of longer context inference on devices with limited GPU memory.

Leveraging ANNS for attention mechanism presents a unique challenge: the out-of-distribution (OOD) problem between query and key vectors. Most ANNS engines operate under the assumption that both query and key vectors are drawn from the same data distribution. However, this assumption does not hold in this context due to the different projection weights for query and key vectors in attention mechanism. The Mahalanobis distance (Mahalanobis, 2018) shows that query vectors deviate more than $10\times$ farther from key vectors compared to that between in-distribution query and key vectors. Unfortunately, the effectiveness of ANNS degrades significantly under OOD problem. In particular, our empirical analysis indicates that maintaining an acceptable level of inference accuracy requires conventional ANNS scanning 30–50% of all key vectors to identify the critical ones, which fails to fully leverage the inherent sparsity of the attention mechanism and impairs the inference latency. To the best of our knowledge, we are the first to identify the challenge of OOD in using ANNS index for attention computation, a factor that is crucial for inference efficiency and accuracy.

In this work, we present RetrievalAttention, an efficient method for accelerating long-context LLM generation. RetrievalAttention employs dynamic sparse attention during token generation, allowing the most critical tokens to emerge from the extensive context data. To address the challenge of OOD, RetrievalAttention proposes a vector index tailored for the attention mechanism, focusing on the distribution of queries rather than keys. This approach allows for the traversal of only a small subset of key vectors (1–3%) to identify the most relevant tokens, yielding accurate attention scores and inference accuracy. In addition, RetrievalAttention reduces GPU memory consumption by retaining a small number of KV vectors in GPU memory following static patterns (e.g., similar to StreamingLLM (Xiao et al., 2024b)) and offloading the majority of KV vectors to CPU memory for index construction. During token generation, RetrievalAttention efficiently retrieves critical tokens using ANNS index on the CPU and merges the partial attention results from both the CPU and GPU. This strategy enables RetrievalAttention to perform attention computation with reduced latency and minimal GPU memory footprint.

We evaluate the accuracy and efficiency of RetrievalAttention on both commodity GPUs (RTX4090) and high-end GPUs (A100) on three long-context LLMs across various long-context benchmarks

*Maximum inner product search can be viewed as similarity search and efficiently solved by ANNS indexes (Morozov & Babenko, 2018).

like ∞ -Bench (Zhang et al., 2024b) and RULER (Hsieh et al., 2024). For the 128K context on the RTX4090 GPU, RetrievalAttention achieves $4.9\times$ and $1.98\times$ decoding-latency reduction compared to the retrieval method based on exact KNN and traditional ANNS index, respectively, while maintaining the same accuracy as full attention. To the best of our knowledge, RetrievalAttention is the first solution that supports running 8B-level models on a single RTX4090 GPU (24GB) with acceptable latency and almost no accuracy degradation.

2 BACKGROUND AND MOTIVATION

2.1 LLM AND ATTENTION OPERATION

In the generation process of the t -th token, the attention operation computes the dot product between the query vector $\mathbf{q}_t \in \mathbb{R}^{1 \times d}$ (where d is the hidden dimension) and the key vectors of all preceding tokens $\mathbf{k}_i \in \mathbb{R}^{1 \times d}$ (for $i \leq t$). This product is scaled by $d^{-\frac{1}{2}}$ and normalized via a `Softmax` function to yield the attention score $a_{t,i}$. These scores then weight the values $\mathbf{v}_i$, resulting in the output $\mathbf{o}_t$.

$$z_i = \frac{\mathbf{q}_t \cdot \mathbf{k}_i^T}{\sqrt{d}}, \quad a_{t,i} = \frac{e^{z_i}}{\sum_{j=1..t} e^{z_j}}, \quad \mathbf{o}_t = \sum_{i=1..t} a_{t,i} \cdot \mathbf{v}_i \quad (1)$$

LLM inference contains two stages: the prefill phase and decoding phase. The prefill phase, which only happens once, takes all tokens of the prompt as input and performs attention with a time-complexity $O(n^2)$. In the decoding (token generation) phase, the newly generated token is added to the input and computes attention scores with same complexity. One common optimization to avoid repetitive calculation is caching past KV states, thereby reducing the complexity to $O(n)$.

2.2 EXPENSIVE LONG-CONTEXT SERVING

Due to the quadratic time complexity of attention operation, serving long-sequence input incurs extremely high costs. Table 1 shows the inference latency of Llama-3-8B without KV cache. When the prompt length reaches 1 million tokens, generating every token requires 1,765 seconds with over 96% of latency spent on attention operations. Although KV cache can reduce the decoding latency, it demands a huge amount of GPU memory for long contexts. As shown in Table 1, 125 GB memory is necessary for storing the KV cache when the context length reaches 1 million tokens, which is far beyond the GPU memory capacity of commodity GPUs such as the RTX4090 (24GB) or even high-end GPUs like A100 (40GB or 80GB). This necessitates either scaling to more GPUs to accommodate the large KV cache (Liu et al., 2024a) or repetitively offloading and reloading the entire KV cache between CPU and GPU memory over PCIe (Sheng et al., 2023), resulting in excessive communication overhead. Neither approach provides an efficient and cost-effective solution for long-context inference on commodity GPUs.

2.3 DYNAMIC AND SPARSE ATTENTION

Corroborating recent work (Xiao et al., 2024b; Li et al., 2024), we observe that attention computation in LLMs exhibits significant sparsity. Despite the large context length, only a small fraction of tokens with the highest attention scores (i.e., $a_{t,i}$ in Equation 1), also known as critical tokens, contribute significantly to the attention output.

We quantify the attention sparsity by calculating the cumulative sum of attention scores of top- k critical tokens. This cumulative sum, called recovery ratio, represents how much of the full attention can be recovered using a small number of critical tokens, with a higher recovery ratio indicating greater sparsity. When generating 20 tokens consecutively based on a prompt of 100,000 tokens, we profile the average recovery ratio of decoding tokens using top-1000 critical tokens in different layers and heads of the model. As shown in the blue curve of Figure 2, by accurately selecting top-1000 critical tokens based on full attention, most attention heads can recover over 90% of the attention scores from the full attention, with an average of 89% across all heads and layers.